【安全】椭圆曲线加密算法(ECC)深入理解

原创 赖small强 于 2021-06-24 23:43:54 发布 阅读量1.6w CC 4.0 BY-SA版权

分类专栏: 安全 文章标签: ECC ECDH Elliptic Curve 非对称加密 椭圆曲线

背景

由于在实际的项目中使用到了基于ECDH的秘钥协商算法，所以对背后基于椭圆曲线密码学(ECC-Elliptic Curve Cryptography)进行了较为深入的学习，本篇文章主要分享一下几点:

- ECC基本介绍
- ECC Keys
- 曲线以及Key的长度
- ECC 相关算法
- 椭圆曲线(Elliptic Curves)
- 有限域上的椭圆曲线
- 有限域上的椭圆曲线: 计算篇
- 将EC 的点乘以一个整数
- ECC上的Private Key, Public Key and the Generator Point
- 基于Python的ECC相关实例讲解

基本概念介绍

ECC基本介绍

ECC的英文全称是: Elliptic Curve Cryptography - 椭圆曲线密码学，是基于有限域的椭圆曲线和复杂的椭圆曲线离散对数.

ECC实现了非对称加密所需要的大部分能力，包括: 加密(encryption)、签名(signatures)、秘钥交换(key exchange).

ECC被认为是RSA密码系统的最好继承者，相对于RSA，ECC的keys更小，在于RSA相同等级下能够更快地生成Key.

ECC Keys

Private keys(私钥)在ECC中使用一个整数来表示，通常是一个256bit的整数. 例如对于一个256bit的ECC private key(16进制编码) 可以表示如下:

bash AI写代码 | 复制

```bash
0x51897b64e85c3f714bba707e867914295a1377a7463a9dae8ea6a8b914246319
```

在ECC中，密钥生成就像在一定范围内安全地生成一个随机整数一样简单，所以这个过程非常的快. ***任何在这个范围的整数都是有效的ECC private keys.***

public key(公钥)在ECC上是ECC points，那什么是ECC points呢？ECC point 代表的是坐落在曲线的整数坐标对{x, y}. 对于public key通常是比private key多一个bit.

曲线以及Key的长度

ECC加密算法可以采用不同的椭圆曲线. 不同的椭圆曲线具备不同的安全等级、不同的性能、不同的key长度，以及可能涉及到的不同底层算法.

ECC 中涉及的椭圆曲线中，非常的出名的有，sepc256k1、Curve25519 等标准化曲线参数集.

ECC keys的长度取决于它采用了哪种椭圆曲线. 例如非常出名的开源项目，OpenSSL、mbedtls等，对于ECC通常默认的private keys 为256bit，但最终取决于开发者使用哪一种椭圆曲线. 常见的有: 92-bit (curve secp192r1), 233-bit (curve sect233k1), 224-bit (curve secp224k1), 256-bit (curves secp256k1 and Curve25519), 283-bit (curve sect283k1), 384-bit (curves p384 and secp384r1), 409-bit (curve sect409r1), 414-bit (curve Curve41417), 448-bit (curve Curve448-Goldilocks), 511-bit (curve M-511), 521-bit (curve P-521), 571-bit (curve sect571k1) . 在mbedtls中对于选择的椭圆曲线，项目中直接提供了对应曲线的椭圆曲线参数.

ECC 相关算法

椭圆曲线密码学基于数学上的椭圆曲线有限域提供了不同使用场景的相关算法.

- ECC 数字签名相关算法: ECDSA、EdDSA
- ECC 加密算法: ECIES、EEECC
- ECC秘钥协商算法: ***ECDH***、X25519等

以上这些算法，都是以private key和public key的形式出现。private key 是一个整数，而public key是对于椭圆曲线上的点(EC Point).

椭圆曲线(Elliptic Curves)

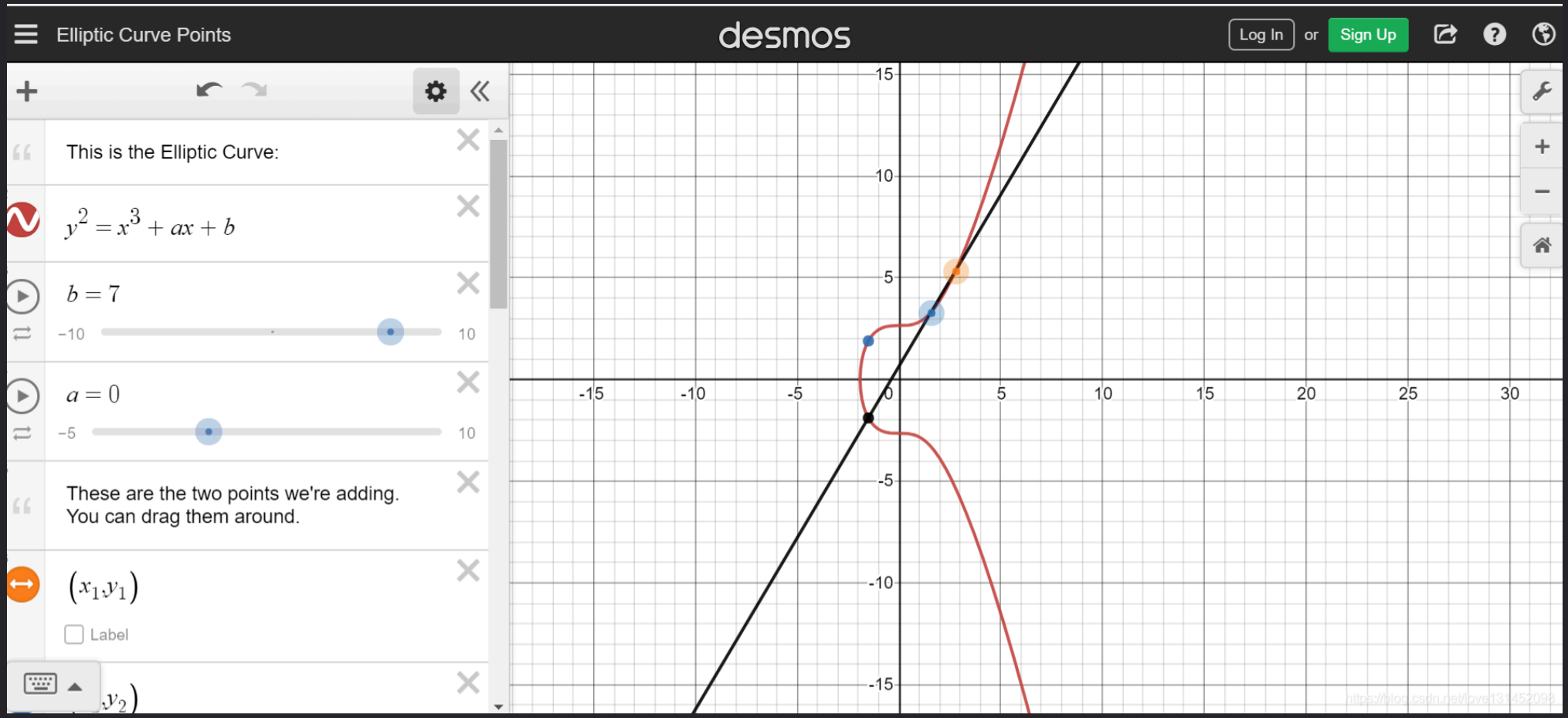
椭圆曲线在数学上，针对密码学做了简化，简化为如下公式:

$$y^2 = x^3 + ax + b$$

例如，对于椭圆曲线: secp256k1, 给出了建议参数, a = 0; b=7,公式转化如下:

$$y^2 = x^3 + 7$$

可视化曲线如下图所示:



PS: 上图生成工具可以通过在线攻击进行生成: https://www.desmos.com/calculator/ialhd71we3

有限域上的椭圆曲线

椭圆曲线密码学是基于有限域上的椭圆曲线. 公式如下:
y2 ≡ x3 + ax + b (mod p) # 由于mod 目前没找到方法生成公式，所以在这里并未转化

针对特定的椭圆曲线secp256k1(Bitcoin curve),
y2 ≡ x3 + 7 (mod p)

和RSA key的范围在整数[0 … p-1]不同，ECC是基于椭圆曲线上有限域的点{x, y}，x和y是在[0…p-1]间的整数.

针对基于有限域的椭圆曲线，存在以下两点规则:

- 存在一组整数坐标: {x, y}, 0<= x < p
- 坐落在椭圆曲线: y2 ≡ x3 + ax + b (mod p)

有限域上的椭圆曲线: 计算篇

针对有限域的椭圆曲线，非常容易计算一个确切的点是否属于该椭圆曲线. 例如，一个带你{x, y}是否属于曲线y2 = x3 + 7(mod 17)可以转化为如下计算看是否满足:

- x3 + 7 -y2 = 0(mod 17)

例如点: 怎么判断{5, 8}是否在该曲线上呢？进行如下计算:
(5**3 + 7 - 8**2) % 17 == 0, 所以{5, 8}在该曲线上.

将EC 的点乘以一个整数

对于两个都在同一椭圆曲线的点相加得到的另外一个点也一定在相同的椭圆曲线上. 这种操作在数学被称作: EC point addition.

针对椭圆曲线有如下规则:
对于一个在一个椭圆曲线上Point(点) G，相乘以一个整数，得到的另一个Point也会在该椭圆曲线上. 而这个相乘的操作是非常快的.

- P = k*G

针对这一个操作内部所包含的公式和转换 哈哈哈 我也还没搞定 在这里我们先忽略 后续进一步学习后再进行分享. 需要记住的一点是知道了k和G计算P是非常快的

ECC椭圆曲线详解 这篇文章中有较为详细的原理解释.

ECC上的Private Key, Public Key and the Generator Point

对于ECC，针对具体的椭圆曲线，存在着确定的(标准给出的建议)点G 成，乘以一个确定的整数k(k就是private key). 可以得到EC point P(public key).

因此,
针对ECC存在如下几点:

- EC 是基于有限域的
- G == generator point (fixed constant, a base point on the EC)
- k == private key (integer)
- P == public key (point)
  针对P = k*G 非常快就能计算出P的值，而计算k的值是非常难的，这也是基于ECC的密码学重点所在. 也就是对于ECC，知道private key很容易计算出public key，而知道public key是很难算出private key(这里面包含的知识是指数和对数的关系).

基于Python的示例演示

EC Point的相乘（教学的例子）

这里需要用到python中的package: tinyec

python　AI写代码 | 复制　▷ 运行

```python
from tinyec.ec import SubGroup, Curve

field = SubGroup(p=17, g=(15, 13), n=18, h=1)
curve = Curve(a=0, b=7, field=field, name='p1707')
print('curve:', curve)

for k in range(0, 25):
    p = k * curve.g
    print(f"{k} * G = ({p.x}, {p.y})")
```

执行结果如下:

bash

AI写代码 | 复制

```bash
curve: "p1707" => y^2 = x^3 + 0x + 7 (mod 17)
0 * G = (None, None)
1 * G = (15, 13)
2 * G = (2, 10)
3 * G = (8, 3)
4 * G = (12, 1)
5 * G = (6, 6)
6 * G = (5, 8)
7 * G = (10, 15)
8 * G = (1, 12)
9 * G = (3, 0)
10 * G = (1, 5)
11 * G = (10, 2)
12 * G = (5, 9)
13 * G = (6, 11)
14 * G = (12, 16)
15 * G = (8, 14)
16 * G = (2, 7)
17 * G = (15, 4)
18 * G = (None, None)
19 * G = (15, 13)
20 * G = (2, 10)
21 * G = (8, 3)
22 * G = (12, 1)
23 * G = (6, 6)
24 * G = (5, 8
```

收起 ^

实际工程会用到的例子

代码如下:

python

AI写代码 | 复制 ▷ 运行

```python
from tinyec import registry

```

```python
curve = registry.get_curve('secp192r1')
print('curve:', curve)

for k in range(0, 10):
    p = k * curve.g
    print(f"{k} * G = ({p.x}, {p.y})")

print("Cofactor =", curve.field.h)

print('Cyclic group order =', curve.field.n)

nG = curve.field.n * curve.g
print(f"n * G = ({nG.x}, {nG.y})")
```

收起 ^

执行结果如下:

bash

AI写代码 | 复制

```bash
curve: "secp192r1" => y^2 = x^3 + 6277101735386680763835789423207666416083908700390324961276x + 2455155546008943817740293915197
0 * G = (None, None)
1 * G = (602046282375688656758213480587526111916698976636884684818, 174050332293622031404857552280219410364023488927386650641)
2 * G = (5369744403678710563432458361254544170966096384586764429448, 5429234379789071039750654906915254128254326554272718558123)
3 * G = (2915109630280678890720206779706963455590627465886103135194, 2946626711558792003980654088990112021985937607003425539581)
4 * G = (1305994880430903997305943738697779408316929565234787837114, 3981863977451150342116987835776121688410789618551673306674)
5 * G = (4102832511167848740189935621365668701106767069367626602 40, 1206654674899825246688205669651974202006189255452737318561)
6 * G = (4008504146453526025173196900303594155799995627910231899946, 3263759301305176906990806636587838100022690095020155627760)
7 * G = (3473339081378406123852871299395262476289672479707038350589, 2152713176906603604200842901176476029776544337891569565621)
8 * G = (1167950611014894512313033362696697441497340081390841490910, 4002177906111215127148483369584652296488769677804145538752)
9 * G = (3176317450453705650283775811228493626776489433309636475023, 4460189377466938476679380385498011517961211807501706220 1)
Cofactor = 1
Cyclic group order = 6277101735386680763835789423176059013767194773182842284081
n * G = (None, None)
```

收起 ^

总结

本文是基于一篇非常优秀的关于ECC的文章结合自己的理解进行输出的.

参考文章链接: Elliptic Curve Cryptography (ECC)

该链接中的系列文章讲述了我们日常开发中常用的关于数据安全相关的算法.

相关文章链接:
https://www.cnblogs.com/Kalafinaian/p/7392505.html